

Analiza kodu aplikacji „Lista filmów”

Omówienie plików, funkcji, przepływów danych i odniesień w kodzie

Dokument został przygotowany na podstawie przekazanych plików Kotlin: MainActivity.kt, Root.kt, MovieViewModel.kt, Screens.kt, AppNavigation.kt, OmdbApi.kt, Movie.kt, MovieDao.kt, MovieDatabase.kt oraz MovieRepository.kt.

Cel dokumentu: wytłumaczyć, za co odpowiada każdy plik, co robią poszczególne funkcje oraz gdzie są wywoływane lub do czego się odnoszą, tak aby można było obronić działanie projektu.

Spis treści

1. Ogólny opis aplikacji i architektury
2. Mapa przepływu danych
3. Opis każdego pliku
4. Indeks funkcji, metod i klas z odniesieniami
5. Scenariusze działania aplikacji
6. Najważniejsze pojęcia do obrony
7. Potencjalne pytania i gotowe odpowiedzi

1. Ogólny opis aplikacji

Aplikacja służy do wyszukiwania filmów po tytule w zewnętrznym API OMDb, zapisywania ich lokalnie oraz dzielenia zapisanych filmów na dwie listy: filmy do obejrzenia i filmy obejrzane. Interfejs jest zbudowany w Jetpack Compose, nawigacja odbywa się przez dolny pasek, dane lokalne są przechowywane w Room, a zapytania sieciowe wykonuje Retrofit.

Warstwa	Pliki	Rola w projekcie
Start aplikacji	MainActivity.kt, Root.kt	Uruchomienie Compose, ustawienie MaterialTheme, utworzenie ViewModelu i przekazanie go do nawigacji.
UI / ekrany	Screens.kt, AppNavigation.kt	Wyświetlanie ekranów, kart filmów, list, formularza wyszukiwania i dolnej nawigacji.
Logika prezentacji	MovieViewModel.kt	Stan UI, obsługa akcji użytkownika, uruchamianie coroutine, komunikacja z repository.
Dane lokalne	Movie.kt, MovieDao.kt, MovieDatabase.kt	Model filmu, zapytania do bazy Room i singleton bazy danych.
Dane z internetu	OmdbApi.kt	Definicja Retrofit API, model odpowiedzi OMDb i konwersja odpowiedzi do Movie.
Warstwa pośrednia	MovieRepository.kt	Łączy ViewModel z bazą danych i API. Ukrywa szczegóły Room/Retrofit przed UI.

2. Mapa przepływu danych

Najprościej można zapamiętać projekt jako przepływ od interfejsu użytkownika do ViewModelu, potem do Repository, a następnie do bazy Room albo API OMDb.

```
Użytkownik
-> ekran Compose: Screens.kt
-> MovieViewModel.kt
-> MovieRepository.kt
    -> MovieDao.kt + MovieDatabase.kt [lokalna baza Room]
    -> OmdbApi.kt [zewnętrzne OMDb API]
-> StateFlow / SearchState
-> ponowne odrysowanie UI w Compose
```

3. Opis plików

MainActivity.kt

Rola: Główny punkt startowy aplikacji Android.

- Klasa MainActivity dziedziczy po ComponentActivity, czyli jest ekranem startowym aplikacji Android.
- Metoda onCreate uruchamia super.onCreate, a następnie wywołuje setContent, czyli przechodzi z klasycznej aktywności Androida do świata Jetpack Compose.
- W środku setContent ustawiany jest MaterialTheme oraz wywoływana funkcja Root().
- Odniesienie: Root jest importowany z com.example.listaфильмов.ui.Root i wywołany w linii 15.

Odniesienia w kodzie: MainActivity.kt: linie 9-18; szczególnie setContent w liniach 13-17.

Root.kt

Rola: Kompozycyjny korzeń interfejsu aplikacji.

- Funkcja Root() jest @Composable, więc tworzy fragment interfejsu w Jetpack Compose.
- Tworzy instancję MovieViewModel przez viewModel(). Dzięki temu wszystkie ekrany korzystają z tego samego źródła stanu.

- Przekazuje movieViewModel do AppNavigation, czyli do elementu zarządzającego ekranami.
- Odniesienie: Root jest wywoływany w MainActivity.kt, a AppNavigation jest wywoływane w Root.kt.

Odniesienia w kodzie: Root.kt: linie 8-15; MainActivity.kt: linia 15; AppNavigation.kt: linie 35-117.

AppNavigation.kt

Rola: Definiuje trasy ekranów i dolny pasek nawigacji.

- sealed class Screen opisuje trzy dostępne ekrany: Watchlist, Search i Watched. Każdy ma własny route, czyli nazwę trasy.
- AppNavigation tworzy navController, obserwuje aktualną trasę i buduje NavigationBar.
- NavigationBarItem sprawdza, czy dana zakładka jest aktualnie wybrana przez porównanie currentRoute == screen.route.
- Kliknięcie zakładki wywołuje navController.navigate(screen.route), więc zmienia ekran.
- NavHost przypisuje trasy do konkretnych funkcji ekranów: WatchlistScreen, SearchScreen i WatchedScreen.

Odniesienia w kodzie: AppNavigation.kt: Screen linie 27-33; AppNavigation linie 35-117; wywołania ekranów linie 104-113.

Screens.kt

Rola: Zawiera widoki Compose: listę do obejrzenia, listę obejrzanych, wyszukiwarkę, listę filmów i kartę filmu.

- WatchlistScreen pobiera viewModel.watchlist jako stan Compose i pokazuje MovieList. Główny przycisk oznacza film jako obejrzany.
- WatchedScreen pobiera viewModel.watched i pokazuje MovieList. Główny przycisk przenosi film z powrotem do listy do obejrzenia.
- SearchScreen trzyma lokalnie wpisany tytuł przez remember/mutableStateOf, obserwuje searchState i pokazuje różne widoki: instrukcję, loader, błąd albo znaleziony film.
- MovieList odpowiada za pokazanie komunikatu pustej listy albo LazyColumn z kartami MovieCard.
- MovieCard pokazuje dane filmu: plakat, tytuł, rok, gatunek, ocenę, opis oraz przyciski akcji.

Odniesienia w kodzie: Screens.kt: WatchlistScreen linie 41-70; WatchedScreen 72-101; SearchScreen 103-175; MovieList 177-214; MovieCard 216-297.

MovieViewModel.kt

Rola: Logika prezentacji i pośrednik między UI a Repository.

- SearchState to sealed interface opisujący możliwe stany wyszukiwarki: Idle, Loading, Success i Error.
- MovieViewModel tworzy MovieRepository, przekazując mu MovieDao pobrane z MovieDatabase.
- watchlist i watched są StateFlow, czyli strumieniami stanu gotowymi do obserwowania przez Compose.
- searchMovie sprawdza, czy wpis nie jest pusty, ustawia Loading, pyta repository o film, a potem ustawia Success albo Error.
- addToWatchlist, markAsWatched, moveToWatchlist i deleteMovie uruchamiają operacje w viewModelScope.launch, żeby nie blokować UI.

Odniesienia w kodzie: MovieViewModel.kt: SearchState linie 16-21; repository 25-27; StateFlow 29-42; searchMovie 44-69; akcje 71-93.

Movie.kt

Rola: Model danych filmu oraz encja Room.

- Adnotacja @Entity(tableName = "movies") mówi Room, że ta klasa odpowiada tabeli movies.
- @PrimaryKey oznacza, że pole id jest kluczem głównym, czyli unikalnym identyfikatorem rekordu.
- Pola title, year, genre, rating, plot i posterUrl opisują dane filmu.
- Pole isWatched decyduje, czy film jest na liście obejrzanych, czy na liście do obejrzenia.

Odniesienia w kodzie: Movie.kt: linie 6-17; używany w MovieDao, MovieRepository, MovieViewModel, Screens i OmdbApi.toMovie.

MovieDao.kt

Rola: Interfejs zapytań do lokalnej bazy Room.

- @Dao mówi Room, że interfejs zawiera operacje bazodanowe.
- insertMovie zapisuje lub nadpisuje film dzięki OnConflictStrategy.REPLACE.
- deleteMovie usuwa film.
- getWatchlist zwraca Flow<List<Movie>> z filmami, gdzie isWatched = 0.
- getWatched zwraca Flow<List<Movie>> z filmami, gdzie isWatched = 1.

Odniesienia w kodzie: MovieDao.kt: linie 10-24; wywoływany w MovieRepository.kt linie 10-36.

MovieDatabase.kt

Rola: Konfiguracja i tworzenie bazy danych Room.

- @Database wskazuje encję i wersję bazy. Tutaj bazę tworzy jedna encja: Movie.
- movieDao() udostępnia DAO, czyli zestaw operacji na bazie.
- Companion object przechowuje singleton INSTANCE, aby aplikacja nie tworzyła wielu instancji bazy.
- @Volatile i synchronized chronią przed problemami wielowątkowości.
- fallbackToDestructiveMigration usuwa i tworzy bazę od nowa przy nieobsługiwanej migracji wersji.

Odniesienia w kodzie: MovieDatabase.kt: @Database linie 8-11; movieDao linia 14; singleton linie 16-34; użycie w MovieViewModel.kt linie 25-27.

OmdbApi.kt

Rola: Warstwa komunikacji z zewnętrznym API OMDb.

- API_KEY przechowuje klucz do OMDb API.
- OmdbApiService definiuje zapytanie GET na ścieżkę "/" z parametrami t oraz apikey.
- OmdbMovieResponse opisuje JSON zwracany przez API. SerializedName mapuje nazwy pól JSON na nazwy pól Kotlin.
- toMovie konwertuje odpowiedź API na lokalny model Movie.
- RetrofitInstance buduje klienta Retrofit z bazowym adresem OMDb i konwerterem Gson.

Odniesienia w kodzie: OmdbApi.kt: API_KEY linia 10; searchMovie linie 12-19; OmdbMovieResponse 21-45; toMovie 47-57; RetrofitInstance 59-67; użycie w MovieRepository.kt linia 14.

MovieRepository.kt

Rola: Warstwa repozytorium łącząca ViewModel, bazę i API.

- Repozytorium przyjmuje MovieDao w konstruktorze, więc nie tworzy bazy bezpośrednio.
- watchlist i watched pochodzą bezpośrednio z dao.getWatchlist() oraz dao.getWatched().
- searchMovie wywołuje RetrofitInstance.api.searchMovie(title), sprawdza Response i zwraca Movie albo null.
- addToWatchlist ustawia isWatched = false i zapisuje film przez DAO.
- markAsWatched ustawia isWatched = true.
- moveToWatchlist ustawia isWatched = false.
- deleteMovie deleguje usunięcie do DAO.

Odniesienia w kodzie: MovieRepository.kt: linie 7-38; używane przez MovieViewModel.kt linie 25-27 oraz 51, 73, 79, 85, 91.

4. Indeks funkcji, klas i odniesień

Element	Plik / linie	Co robi	Gdzie jest używany /
---------	--------------	---------	----------------------

			odniesienie
MainActivity.onCreate	MainActivity.kt 10-18	Uruchamia Compose przez setContent, ustawia MaterialTheme i startuje Root().	System Android wywołuje onCreate przy starcie aktywności; Root() linia 15.
Root()	Root.kt 8-15	Tworzy MovieViewModel przez viewModel() i przekazuje go do AppNavigation.	Wywołanie w MainActivity.kt 15; przekazuje stan do AppNavigation.kt.
Screen	AppNavigation.kt 27-33	Zamknięta lista tras ekranów.	Używany w AppNavigation: lista screens 43-47, NavigationBar 52-95, NavHost 104-113.
AppNavigation()	AppNavigation.kt 35-117	Buduje dolną nawigację i NavHost.	Wywoływany w Root.kt 12-14.
WatchlistScreen()	Screens.kt 41-70	Ekran filmów do obejrzenia.	Trasa Screen.Watchlist w AppNavigation.kt 104-106.
WatchedScreen()	Screens.kt 72-101	Ekran filmów obejrzanych.	Trasa Screen.Watched w AppNavigation.kt 112-114.
SearchScreen()	Screens.kt 103-175	Ekran wyszukiwania filmów.	Trasa Screen.Search w AppNavigation.kt 108-110.
MovieList()	Screens.kt 177-214	Pokazuje pustą listę albo LazyColumn z MovieCard.	Wywołany w WatchlistScreen 57-68 i WatchedScreen 88-99.
MovieCard()	Screens.kt 216-297	Prezentuje dane pojedynczego filmu i przyciski akcji.	Wywołany w MovieList 201-210 oraz SearchScreen 163-170.
SearchState	MovieViewModel.kt 16-21	Modeluje stany wyszukiwania.	Używany w SearchScreen.kt 146-171 i w MovieViewModel searchState.
MovieViewModel.searchMovie()	MovieViewModel.kt 44-69	Obsługuje wyszukiwanie: Loading -> Success/Error.	Wywołanie z SearchScreen.kt 137-140.
MovieViewModel.addToWatchlist()	MovieViewModel.kt 71-75	Dodaje film jako nieobejrzany.	Wywołanie z SearchScreen.kt 166-168.
MovieViewModel.markAsWatched()	MovieViewModel.kt 77-81	Ustawia film jako obejrzany.	Wywołanie z WatchlistScreen.kt 62-64.
MovieViewModel.moveToWatchlist()	MovieViewModel.kt 83-87	Cofa film do listy do obejrzenia.	Wywołanie z WatchedScreen.kt 93-95.
MovieViewModel.deleteMovie()	MovieViewModel.kt 89-93	Usuwa film z bazy.	Wywołanie z WatchlistScreen.kt 65-67 i WatchedScreen.kt 96-98.
Movie	Movie.kt 6-17	Encja Room i model danych aplikacji.	Używany w DAO, Repository, ViewModel, UI oraz toMovie().
MovieDao.insertMovie()	MovieDao.kt 13-14	Dodaje lub zastępuje film.	Wywoływany w MovieRepository.kt 24, 28, 32.
MovieDao.deleteMovie()	MovieDao.kt 16-17	Usuwa film.	Wywoływany w MovieRepository.kt 36.
MovieDao.getWatchlist()	MovieDao.kt 19-20	Zwraca Flow filmów z isWatched = 0.	Używany w MovieRepository.kt 10; obserwowany przez WatchlistScreen.
MovieDao.getWatched()	MovieDao.kt 22-23	Zwraca Flow filmów z isWatched = 1.	Używany w MovieRepository.kt 11; obserwowany przez WatchedScreen.
MovieDatabase.getDatabase()	MovieDatabase.kt 20-33	Tworzy lub zwraca singleton bazy Room.	Wywoływany w MovieViewModel.kt 25-27.
OmdbApiService.searchMovie()	OmdbApi.kt 14-18	Zapytanie HTTP GET do OMDb.	Wywoływane w MovieRepository.kt 14.
OmdbApiResponse.toMovie()	OmdbApi.kt 47-57	Mapuje odpowiedź API na encję Movie.	Wywoływane w MovieRepository.kt 17.
RetrofitInstance.api	OmdbApi.kt 59-67	Leniwie tworzy klienta Retrofit.	Używany w MovieRepository.kt 14.

5. Scenariusze działania aplikacji

5.1. Start aplikacji

8. Android uruchamia MainActivity i metodę onCreate.

9. setContent przełącza aplikację na interfejs Compose.
10. MaterialTheme ustawia wygląd komponentów Material 3.
11. Root tworzy MovieViewModel i przekazuje go do AppNavigation.
12. AppNavigation startuje od trasy watchlist, więc pierwszym ekranem jest lista filmów do obejrzenia.

5.2. Wyszukiwanie filmu

13. Użytkownik wpisuje tytuł w OutlinedTextField w SearchScreen.
14. Po kliknięciu przycisku Szukaj wywoływane jest viewModel.searchMovie(title).
15. ViewModel ustawia SearchState.Loading, co powoduje pokazanie CircularProgressIndicator.
16. Repository wykonuje zapytanie Retrofit do OMDb API.
17. Jeśli OMDb zwróci Response == "True", odpowiedź jest konwertowana do Movie przez toMovie().
18. ViewModel ustawia SearchState.Success(movie), a UI pokazuje MovieCard.
19. Jeśli filmu nie ma lub pojawi się błąd połączenia, ViewModel ustawia SearchState.Error z komunikatem.

```
SearchScreen -> MovieViewModel.searchMovie() -> MovieRepository.searchMovie() ->
RetrofitInstance.api.searchMovie() -> OmdbMovieResponse.toMovie() -> SearchState.Success/Error ->
SearchScreen
```

5.3. Dodanie filmu do listy do obejrzenia

20. Na karcie wyniku wyszukiwania użytkownik klika przycisk dodania do listy.
21. SearchScreen wywołuje viewModel.addToWatchlist(state.movie).
22. ViewModel uruchamia coroutine w viewModelScope.
23. Repository zapisuje kopię filmu z isWatched = false.
24. DAO wykonuje insertMovie, a ponieważ strategia konfliktu to REPLACE, film zostanie dodany albo nadpisany.
25. Flow z getWatchlist emituje nową listę, a WatchlistScreen odświeża się automatycznie.

5.4. Oznaczenie filmu jako obejrzany

26. Na ekranie WatchlistScreen użytkownik klika główny przycisk na MovieCard.
27. WatchlistScreen wywołuje viewModel.markAsWatched(movie).
28. Repository zapisuje film z isWatched = true.
29. Ten sam rekord w tabeli movies zmienia status.
30. Film znika z listy do obejrzenia, bo getWatchlist filtruje isWatched = 0.
31. Film pojawia się na liście obejrzanych, bo getWatched filtruje isWatched = 1.

5.5. Usunięcie filmu

32. Użytkownik klika przycisk Usuń na MovieCard.
33. Ekran wywołuje viewModel.deleteMovie(movie).
34. ViewModel przekazuje operację do Repository.
35. Repository wywołuje dao.deleteMovie(movie).
36. Room usuwa rekord, a obserwowane Flow emituje zaktualizowaną listę.

6. Najważniejsze pojęcia do obrony

Pojęcie	Jak wytłumaczyć na obronie	Gdzie w kodzie
Jetpack Compose	Deklaratywny sposób budowania UI. Funkcje @Composable opisują, jak ekran ma wyglądać dla aktualnego stanu.	Root.kt, Screens.kt, AppNavigation.kt
StateFlow	Strumień stanu. Gdy baza lub ViewModel zmieni dane, UI może się automatycznie odświeżyć.	MovieViewModel.kt 29-42; Screens.kt 46, 77, 109
collectAsState	Łączy Flow/StateFlow z Compose. Gdy stan się zmieni, ekran zostanie przerysowany.	Screens.kt 46, 77, 109
ViewModel	Przechowuje logikę i stan UI niezależnie od samego ekranu Compose.	MovieViewModel.kt

Repository	Oddziela ViewModel od szczegółów bazy i API. ViewModel nie musi wiedzieć, jak działa Retrofit albo Room.	MovieRepository.kt
Room	Biblioteka ORM do lokalnej bazy SQLite. Używa encji Movie, DAO i klasy MovieDatabase.	Movie.kt, MovieDao.kt, MovieDatabase.kt
Retrofit	Biblioteka do wykonywania zapytań HTTP do API. Tutaj pobiera dane filmu z OMDb.	OmdbApi.kt
Coroutine / suspend	Operacje bazodanowe i sieciowe są wykonywane asynchronicznie, żeby nie blokować interfejsu.	MovieViewModel.kt i funkcje suspend w DAO/Repository/API
sealed interface SearchState	Bezpieczny model stanów wyszukiwania. Ekran wie dokładnie, jakie sytuacje może obsłużyć.	MovieViewModel.kt 16-21; Screens.kt 146-171
Room OnConflictStrategy.REPLACE	Jeśli film z tym samym id już istnieje, zostanie zastąpiony. Dzięki temu zmiana isWatched działa jak aktualizacja.	MovieDao.kt 13-14; MovieRepository.kt 24, 28, 32

7. Potencjalne pytania i gotowe odpowiedzi

Dlaczego użyto ViewModelu?

Żeby oddzielić logikę aplikacji od UI. Ekran Compose tylko pokazuje stan i przekazuje akcje użytkownika, a ViewModel decyduje, co zrobić dalej.

Po co jest Repository?

Repository jest warstwą pośrednią. Dzięki niemu ViewModel nie komunikuje się bezpośrednio ani z Room, ani z Retrofit. Kod jest czytelniejszy i łatwiejszy do testowania.

Dlaczego w DAO zwracany jest Flow?

Flow pozwala obserwować zmiany w bazie. Gdy film zostanie dodany, usunięty albo zmieni status, UI może automatycznie pokazać aktualną listę.

Jak aplikacja rozróżnia filmy obejrzone i nieobejrzone?

Przez pole isWatched w encji Movie. W DAO getWatchlist wybiera rekordy z isWatched = 0, a getWatched z isWatched = 1.

Co się dzieje po kliknięciu „Oznacz jako obejrzany”?

WatchlistScreen wywołuje markAsWatched w ViewModelu. Repository zapisuje kopię filmu z isWatched = true. Room aktualizuje rekord, a listy odświeżają się przez Flow.

Co robi SearchState?

Opisuje stan wyszukiwarki: bezczynność, ładowanie, sukces lub błąd. Dzięki temu SearchScreen wie, czy pokazać instrukcję, loader, komunikat błędu czy kartę filmu.

Jak działa pobieranie filmu z internetu?

Repository wywołuje RetrofitInstance.api.searchMovie(title). Retrofit wysyła GET do OMDb, Gson mapuje JSON na OmdbMovieResponse, a toMovie zamienia odpowiedź na Movie.

Dlaczego użyte są coroutine?

Zapytania sieciowe i operacje bazy mogą trwać. Coroutine uruchomione w viewModelScope pozwalają wykonać je poza głównym wątkiem UI.

Co robi fallbackToDestructiveMigration?

Przy zmianie wersji bazy bez migracji Room może usunąć starą bazę i utworzyć nową. To proste w projekcie uczelnianym, ale w aplikacji produkcyjnej zwykle przygotowuje się migracje.

Czy klucz API w kodzie to dobre rozwiązanie?

Do projektu demonstracyjnego jest proste, ale w produkcji klucza API nie powinno się trzymać jawnie w repozytorium. Lepiej użyć konfiguracji build lub backendu pośredniczącego.

8. Uwagi techniczne i miejsca warte wspomnienia

- W przekazanych plikach nie ma strings.xml ani build.gradle, więc dokument opisuje tylko kod Kotlin. Odwołania `R.string.*` oznaczają zasoby tekstowe aplikacji, które powinny znajdować się w plikach resources.
- Klucz OMDb API jest wpisany bezpośrednio w `OmdbApi.kt`. Na obronie można powiedzieć, że to uproszczenie projektu, a w produkcji należałoby go zabezpieczyć.
- Brak osobnej funkcji update w DAO nie jest problemem, ponieważ insertMovie z `OnConflictStrategy.REPLACE` działa jak nadpisanie rekordu o tym samym kluczu głównym.
- MovieCard przyjmuje onDeleteClick jako nullable. Dzięki temu w wynikach wyszukiwania nie ma przycisku usuwania, a na listach lokalnych jest.
- Room filtruje listy po polu isWatched, więc jedna tabela movies obsługuje dwa widoki aplikacji.
- SearchScreen ma lokalny stan pola tekstowego title, bo wpisywany tytuł nie musi być przechowywany w bazie ani w ViewModelu.

9. Krótkie podsumowanie do powiedzenia na obronie

Projekt jest aplikacją Android do zarządzania listą filmów. Interfejs został zrobiony w Jetpack Compose, a ekrany są przełączane przez Navigation Compose. Dane o filmie są wyszukiwane w OMDb API przez Retrofit, następnie mapowane na lokalny model Movie i zapisywane w bazie Room. ViewModel trzyma stany dla UI oraz uruchamia operacje asynchroniczne w viewModelScope. Repository oddziela logikę ViewModelu od szczegółów dostępu do API i bazy. Dwie listy - do obejrzenia i obejrzone - są realizowane przez jedno pole isWatched w tabeli movies, a UI automatycznie odświeża się dzięki Flow i StateFlow.

10. Lista analizowanych plików

Plik	Najważniejsze elementy
MainActivity.kt	onCreate, setContent, MaterialTheme, Root
Root.kt	Root, viewModel, AppNavigation
AppNavigation.kt	Screen, AppNavigation, NavigationBar, NavHost
Screens.kt	WatchlistScreen, WatchedScreen, SearchScreen, MovieList, MovieCard
MovieViewModel.kt	SearchState, MovieViewModel, searchMovie, operacje na filmach
Movie.kt	Encja Movie, pola filmu, isWatched
MovieDao.kt	insertMovie, deleteMovie, getWatchlist, getWatched
MovieDatabase.kt	RoomDatabase, singleton, getDatabase
OmdbApi.kt	OmdbApiService, OmdbApiResponse, toMovie, RetrofitInstance
MovieRepository.kt	searchMovie, addToWatchlist, markAsWatched, moveToWatchlist, deleteMovie